STUDENT REFLECTION & LEARNING NOTES

Name: Devesh Yadav | Roll Number: 230101052

Course: CS3081 Cloud Computing | Date: November 2026

GitHub Repository: https://github.com/Devesh-Yadav22/llm-distillation-k8s-cloud

---

1. Key Learnings & Conceptual Understanding

During this mini-project, I gained a deep, hands-on understanding of bridging modern natural language processing with cloud-native infrastructure. Deploying large generative AI models (such as 3B+ parameter LLMs) on low-end local or edge devices is computationally prohibitive due to severe RAM, memory bandwidth, and thermal bottlenecks. Knowledge distillation emerged as a pragmatic engineering solution: by transferring task-specific capabilities into a lightweight 70M-parameter model, we preserved structural 4-key JSON command parsing accuracy while shrinking memory requirements by over 95%.

In Kubernetes, I learned the critical difference between liveness (/healthz) and readiness (/readyz) probes. Decoupling model weights from the container image requires downloading checkpoints from S3 storage (SeaweedFS) at startup. Without a proper readiness probe, Kubernetes immediately routes ingress traffic to new pods before model tensors are resident in RAM, causing widespread HTTP 500/503 bursts. By keeping readiness unready until initialization finishes, Kubernetes guarantees zero request drops during pod recreation.

In networking, designing stateless Network Access Control Lists (NACLs) reinforced that stateless firewalls retain no connection memory. Unlike AWS Security Groups, a packet's outbound path and its returning response are evaluated independently. Allowing web traffic on ports 80/443 is futile unless return rules explicitly open client ephemeral ports (1024-65535).

2. Major Challenges Faced & Practical Solutions

Challenge 1: Docker Image Size Exploding Beyond 2 GB Limit - Problem: When first installing PyTorch in the inference service Dockerfile, standard 'pip install torch' pulled hundreds of megabytes of CUDA 12 and cuDNN shared libraries, generating a bloated 3.6 GB Docker image that failed the 2 GB assignment requirement. - Solution: I researched PyTorch distribution wheels and configured pip to pull exclusively from the official CPU wheel repository (https://download.pytorch.org/whl/cpu). This eliminated all GPU drivers, reducing the final container image size to 980 MB.

Challenge 2: Teacher Hallucinations in Slot Extraction - Problem: The teacher model (Phi-3.5) occasionally canonicalized natural language expressions (e.g. converting 'seven in the evening' to '19:00' or inventing dates). Rule 1 explicitly states that slots must be extracted verbatim without conversion. - Solution: In worker/clean.py, I developed a strict substring validation check (Rule d) that verifies whether every extracted non-null slot string exists verbatim in the original command text. This intercepted 31 hallucinated records, ensuring that the student model only fine-tuned on strictly conforming targets.

Challenge 3: Preventing Dropped Requests During Rolling Updates - Problem: Triggering rolling updates under Locust load initially generated intermittent 502/503 errors because Traefik sent requests to terminating replicas before new replicas finished loading model tensors. - Solution: I tuned the Deployment strategy with maxSurge=1 and maxUnavailable=0. Combined with a readiness probe on /readyz, Kubernetes ensured the new replica was 100% warm and answering HTTP 200 before removing the older pod, slashing rollout failures to just 0.21% (well below the 0.5% threshold).

3. Observations, Empirical Findings & Future Improvements

Key Observations & Empirical Findings: 1. Distillation Efficiency: Pythia-70M learned strict JSON syntax remarkably fast. Prior to fine-tuning, the untrained base model achieved 0.0% JSON validity. After 3 epochs of sequence-level distillation on 1,500 teacher examples, JSON validity surged to 99.0%, and full-frame accuracy reached 78.0%. This demonstrated that even tiny 70M models can effectively learn structured grammar. 2. Resource Headroom: Profiling 50 sequential requests under a 1 CPU / 1 GiB limit revealed peak memory consumption of 684.2 MiB (66.8% of capacity). The service ran with zero Out-Of-Memory (OOM) kills, confirming feasibility on resource-constrained laptops. 3. Cost Feasibility: Based on the hypothetical cloud rate card, serving 1,000 requests costs only INR 0.26. Under our 99% SLO, we consumed only 29.3% of our error budget during intensive 2-minute Locust stress testing.

What Could Have Been Done Better & Proposed Improvements: 1. Active Distillation / Student Error Correction: Sequence-level distillation is prone to exposure bias. If the student makes an error early in the generation, it cannot recover. In future iterations, implementing an on-policy reinforcement learning or DPO correction loop would significantly boost edge-case resilience. 2. Inference Engine Acceleration: The Python FastAPI / PyTorch CPU runtime generates tokens sequentially. Integrating ONNX Runtime or OpenVINO with INT8 quantization would yield an estimated 3x to 4x throughput improvement on standard x86 CPU cores. 3. Multi-Node Resilience: Because k3d executes on a single physical host, hardware multi-machine redundancy cannot be demonstrated. Deploying across a real multi-node cloud provider with Terraform would validate true cross-zone disaster recovery.

Student Declaration & Signature:

I hereby declare that this project represents my own authentic work, code, and reflection notes.

Signed: Devesh Yadav | Roll Number: 230101052 | Date: November 2026